Failure (an endless loop, and a real pause/resume) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `loop_limit_interrupt_practice.py`: watching an endless loop actually hit its limit, and a paused graph actually resume across a real process restart, is the difference between trusting these mechanisms and just believing the docs about them. **If not:** the Build Task's own checkpointer choice (`MemorySaver` vs. `SqliteSaver`) would be a guess instead of something you watched fail and then fixed yourself.

This exercise has 2 separate halves — the endless loop, and the checkpoint/interrupt/resume cycle. Each version below covers both. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

Part 1 — the endless loop, stopped by a low recursion limit:

```
# loop_limit_interrupt_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.errors import GraphRecursionError


class LoopState(TypedDict):
    count: int


def node_a(state):
    return {"count": state["count"] + 1}


def node_b(state):
    return {"count": state["count"] + 1}
```

```
builder = StateGraph(LoopState)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
builder.add_edge("node_b", "node_a")

graph = builder.compile()

try:
    graph.invoke({"count": 0}, config={"recursion_limit": 5})
except GraphRecursionError as e:
    print(f"Stopped on purpose after hitting the recursion limit: {e}")
```

Expected output:

```
Stopped on purpose after hitting the recursion limit: Recursion
limit of 5 reached without hitting a stop condition. You can
increase the limit by setting the `recursion_limit` config key.
```

(shown wrapped onto three lines just to fit the page — really one line of output; the exact wording can also vary slightly by LangGraph version. The reliable part is the exception type, `GraphRecursionError`, and that it fires instead of the process hanging forever.)

Part 2 — checkpoint, interrupt, resume:

```
# loop_limit_interrupt_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command


class ApprovalState(TypedDict):
    task: str
    approved: bool


def do_task(state):
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
    return {"approved": decision}


builder = StateGraph(ApprovalState)
builder.add_node("do_task", do_task)
builder.add_edge(START, "do_task")
```

```
builder.add_edge("do_task", END)

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "run-1"}}
starting_state = {"task": "send an email", "approved": False}

first_result = graph.invoke(starting_state, config=config)
print("after first invoke:", first_result)

second_result = graph.invoke(Command(resume=True), config=config)
print("after resume:", second_result)
```

Expected output:

```
after first invoke: {'task': 'send an email', 'approved': False,
'__interrupt__': [Interrupt(value={'question': 'Approve running:
send an email?'}, ...)]}
after resume: {'task': 'send an email', 'approved': True}
```

(the first line above is shown wrapped onto three lines just to fit the page — really one line of output) The first `invoke()` runs `do_task`, which calls `interrupt(...)` and pauses right there — the graph never reaches `END` on that call, and the returned state carries the interrupt's payload instead of a finished result. The second `invoke()`, using `Command(resume=True)` and the **same** `thread_id`, picks `do_task` back up from exactly where `interrupt()` paused it — `decision` becomes `True`, the function finishes, and `approved` ends up `True` in the final state.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints, and reading the interrupt payload properly

Part 1:

```
# loop_limit_interrupt_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.errors import GraphRecursionError


class LoopState(TypedDict):
    count: int


def node_a(state: LoopState) -> dict:
    return {"count": state["count"] + 1}


def node_b(state: LoopState) -> dict:
    return {"count": state["count"] + 1}


builder = StateGraph(LoopState)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
builder.add_edge("node_b", "node_a")

graph = builder.compile()

try:
    final_state = graph.invoke({"count": 0}, config={"recursion_limit": 5})
    # why: unreachable by design -- there's no exit edge from this loop
    print(f"Somehow finished: {final_state}")
except GraphRecursionError:
    print("Confirmed: an exit-less loop fails loudly, not silently.")
```

Expected output:

```
Confirmed: an exit-less loop fails loudly, not silently.
```

Part 2:

```
# loop_limit_interrupt_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command


class ApprovalState(TypedDict):
    task: str
    approved: bool


def do_task(state: ApprovalState) -> dict:
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
    return {"approved": decision}


builder = StateGraph(ApprovalState)
builder.add_node("do_task", do_task)
builder.add_edge(START, "do_task")
builder.add_edge("do_task", END)

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "run-1"}}
starting_state = {"task": "send an email", "approved": False}

paused_state = graph.invoke(starting_state, config=config)

pending_interrupts = paused_state.get("__interrupt__", [])
if pending_interrupts:
    question = pending_interrupts[0].value["question"]
    print(f"Graph paused, asking: {question}")
else:
    print("Graph did not pause -- something's wrong with the interrupt wiring")

resumed_state = graph.invoke(Command(resume=True), config=config)
print(f"Final state after resume: {resumed_state}")
```

Expected output:

```
Graph paused, asking: Approve running: send an email?
Final state after resume: {'task': 'send an email', 'approved': True}
```

Difference from Basic: Part 1 adds a check that the graph genuinely never finishes on its own (the `try` block's success branch is unreachable by design, which is itself worth confirming). Part 2 reads the actual interrupt payload out of `paused_state["__interrupt__"]` and prints the real question instead of just printing the whole raw state dict — this is what a real approval UI would do: show the human the *question*, not a debug dump.

Approach 2 — proving `MemorySaver` does NOT survive a real restart

Story: proving pause/resume "works" by calling `invoke()` twice in the same script, with the same `MemorySaver` object still sitting in a variable, doesn't actually prove state survives a real restart — it only proves the object in memory still has it. **If not:** the Build Task's `graph.py` would ship with an untested assumption about what its checkpointer actually needs to survive.

This is 2 separate scripts, run one after the other, with the process exiting completely in between — not 2 function calls in the same script.

`pause_script.py`:

```
# pause_script.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt


class ApprovalState(TypedDict):
    task: str
    approved: bool


def do_task(state: ApprovalState) -> dict:
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
    return {"approved": decision}


builder = StateGraph(ApprovalState)
builder.add_node("do_task", do_task)
builder.add_edge(START, "do_task")
builder.add_edge("do_task", END)
```

```
checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "run-1"}}

result = graph.invoke(
    {"task": "send an email", "approved": False}, config=config
)
print("Paused. Now exit this process entirely and run resume_script.py "
      "separately.")
print(result)
```

`resume_script.py` (run as a brand-new `python resume_script.py`, after `pause_script.py` has already exited):

```
# resume_script.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command


class ApprovalState(TypedDict):
    task: str
    approved: bool


def do_task(state: ApprovalState) -> dict:
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
    return {"approved": decision}


builder = StateGraph(ApprovalState)
builder.add_node("do_task", do_task)
builder.add_edge(START, "do_task")
builder.add_edge("do_task", END)

# a brand-new MemorySaver -- this process never saw the first invoke() at all
checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "run-1"}}

resumed_state = graph.invoke(Command(resume=True), config=config)
print(resumed_state)
```

Expected output from `resume_script.py`:

```
Traceback (most recent call last):
  ...
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
KeyError: 'task'
```

This is the point, not a mistake to fix: `MemorySaver` really does keep its data only in the Python process that created it. A genuinely separate process has a genuinely empty checkpointer: there is no saved pause for `run-1`, so LangGraph starts the graph from the beginning with an **empty** state — and `do_task` crashes reading `state['task']`. Nothing was resumed — proving hands-on that `MemorySaver` is a development/testing tool, not something that survives an actual restart.

Approach 3 — a checkpointer that actually persists, using SQLite

Story: the one substitution that fixes Approach 2's failure — `SqliteSaver` instead of `MemorySaver` — is the exact seam the Build Task needs, so `graph.py` can swap checkpointers at the call site without touching the graph itself. **If not:** the Build Task's skeleton would hard-code `MemorySaver()`, and the day Project 3 needs a checkpointer that survives a restart, that would mean editing the graph's own code instead of just the caller.

Three files this time: the graph lives in one module, and the two scripts import it — so both processes build exactly the same graph. `SqliteSaver` needs `pip install langgraph-checkpoint-sqlite`.

```
# approval_graph.py
from typing import TypedDict

from langgraph.graph import END, START, StateGraph
from langgraph.types import interrupt


class ApprovalState(TypedDict):
    task: str
    approved: bool


def do_task(state: ApprovalState) -> dict:
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
    return {"approved": decision}


def build_approval_graph(checkpointer):
    builder = StateGraph(ApprovalState)
    builder.add_node("do_task", do_task)
    builder.add_edge(START, "do_task")
```

```
    builder.add_edge("do_task", END)
    return builder.compile(checkpointer=checkpointer)
```

```
# pause_script.py — run first; it pauses, saves to disk, and exits
from langgraph.checkpoint.sqlite import SqliteSaver

from approval_graph import build_approval_graph

CONFIG = {"configurable": {"thread_id": "run-1"}}

with SqliteSaver.from_conn_string("checkpoints.sqlite") as checkpointer:
    graph = build_approval_graph(checkpointer)
    graph.invoke({"task": "send an email", "approved": False}, config=CONFIG)
    print("Paused, saved to checkpoints.sqlite. Safe to exit now.")
```

```
# resume_script.py — run second, as a brand-new process
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command

from approval_graph import build_approval_graph

CONFIG = {"configurable": {"thread_id": "run-1"}}

with SqliteSaver.from_conn_string("checkpoints.sqlite") as checkpointer:
    graph = build_approval_graph(checkpointer)
    snapshot = graph.get_state(CONFIG)
    print(f"waiting at: {snapshot.next}")
    print(f"saved state: {snapshot.values}")
    resumed_state = graph.invoke(Command(resume=True), config=CONFIG)
    print(f"after resume: {resumed_state}")
```

Run them one after the other — the first process exits completely before the second starts:

```
python pause_script.py
python resume_script.py
```

Expected output:

```
Paused, saved to checkpoints.sqlite. Safe to exit now.
```

```
waiting at: ('do_task',)
saved state: {'task': 'send an email', 'approved': False}
```

```
after resume: {'task': 'send an email', 'approved': True}
```

Same graph, same interrupt, same `Command(resume=True)` call — the only change is `SqliteSaver.from_conn_string("checkpoints.sqlite")`, which writes each checkpoint to a file on disk. `graph.get_state(config)` reads that saved checkpoint without running anything: `.next` is the node waiting to run (an empty tuple `()` when nothing is paused), and `.values` is the saved state. That's how a second process knows there's something to resume before it resumes it.

Difference from Approach 1, and between Approaches 2/3: Approach 1 proves pause/resume works within one running script, using `MemorySaver`. Approach 2 deliberately tries the same thing across 2 truly separate processes and shows it fails — the checkpointer's data was only ever in that first process's RAM. Approach 3 fixes the actual problem Approach 2 exposes, by swapping in `SqliteSaver`, a checkpointer backed by a real file — proving, with two real scripts, that the exact same graph and interrupt code now survives a restart — and `get_state()` shows what's waiting before you resume it.

Which one should you actually use? `MemorySaver` for local development and this document's exercises — it's fast, needs no setup, and is exactly right for testing a graph's logic. The moment a pause needs to survive past the current process — a real human approval workflow, a server that might restart, anything actually going to production — swap to a real persisted checkpointer (`SqliteSaver` for something simple and local, `PostgresSaver` for a real deployed service). Approach 2's failure is worth actually watching once, the same way the endless loop in Part 1 is — so "why didn't my resume work" never becomes a confusing surprise in a real project.